

Métrologie Système, Exploration Matérielle et CIM

Ce document détaille l'accès aux sous-systèmes matériels et logiciels de Windows via le standard CIM (*Common Information Model*), l'optimisation des requêtes réseau par filtrage WQL, l'arithmétique de normalisation des grandeurs physiques et la conception d'outils d'inventaire d'infrastructure fiables et résilients.

1. Fondations Architecturales : De WMI à CIM

Pendant plus de deux décennies, l'administration bas-niveau sous Windows reposait sur **WMI** (*Windows Management Instrumentation*), l'implémentation propriétaire par Microsoft du standard WBEM (*Web-Based Enterprise Management*).

Bien qu'éprouvée, la pile WMI historique reposait sur une architecture réseau contraignante qui a conduit Microsoft à la déclarer obsolète et à la remplacer par la pile moderne **CIM**.

2. Cartographie et Espaces de Noms (root/cimv2)

Le référentiel d'instrumentation de Windows est structuré sous forme d'une base de données hiérarchique appelée le référentiel CIM. Ce référentiel est découpé en **espaces de noms** (*namespaces*).

L'espace de noms par défaut, qui héberge la quasi-totalité des sondes utiles à l'audit système, est : (root/cimv2)

2.1 Découverte interactive des classes (Get-CimClass)

Pour identifier les classes capables de répondre à un besoin d'inventaire sans consulter la documentation Internet, l'administrateur interroge l'annuaire local à l'aide de motifs génériques (*) :

```
# Identifier les classes relatives au stockage et aux volumes
Get-CimClass -ClassName *LogicalDisk*
```

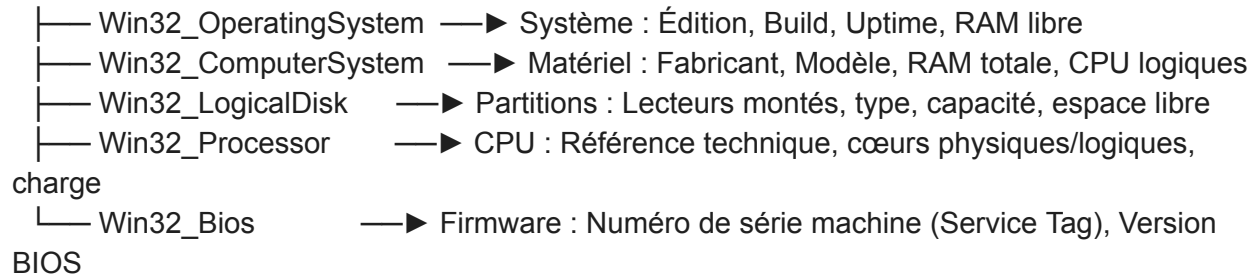
```
# Identifier les classes relatives au processeur ou à la mémoire
Get-CimClass -ClassName *Processor*
Get-CimClass -ClassName *PhysicalMemory*
```

```
# Lister les propriétés exposées par une classe précise
(Get-CimClass -ClassName Win32_OperatingSystem).CimClassProperties.Name
```

2.2 Les Cinq Classes Piliers de l'Audit d'Infrastructure

Dans un environnement de production, 95% des besoins d'inventaire matériel et système sont couverts par 5 classes fondamentales :

[Référentiel root/cimv2]



Détail des propriétés stratégiques :

1. Win32_OperatingSystem :

- Caption : Nom complet du système d'exploitation avec l'édition (ex: *Microsoft Windows Server 2022 Datacenter*).
- Version / BuildNumber : Version exacte du noyau (indispensable pour l'audit des parcs hétérogènes).
- LastBootUpTime : Horodatage précis du dernier redémarrage de la machine.
- FreePhysicalMemory : Quantité de RAM immédiatement disponible en **Kilo-octets (Ko)**.
- TotalVisibleMemorySize : Quantité de mémoire adressable par le noyau en **Kilo-octets (Ko)**.

2. Win32_ComputerSystem :

- Manufacturer : Constructeur physique (ex: *Dell Inc.*, *HP*, *Lenovo*) ou fournisseur d'hyperviseur (ex: *VMware, Inc.*, *Microsoft Corporation*).
- Model : Modèle de la carte mère, du châssis ou de la VM (ex: *PowerEdge R640*, *VMware Virtual Platform*).
- TotalPhysicalMemory : Mémoire vive totale installée en **Octets (Bytes)**.
- NumberOfLogicalProcessors : Nombre total de threads de calcul disponibles.

3. Win32_LogicalDisk :

- DeviceID : Lettre d'affectation du lecteur (ex: C:, D:).
- DriveType : Indicateur numérique qualifiant la nature de la partition (fixe, réseau, amovible).

- Size : Capacité totale de la partition en **Octets (Bytes)**.
- FreeSpace : Espace résiduel disponible en **Octets (Bytes)**.
- 4. **Win32_Processor** :
 - Name : Référence commerciale complète du processeur.
 - NumberOfCores : Nombre de cœurs d'exécution physiques.
 - NumberOfLogicalProcessors : Nombre de cœurs logiques (tenant compte de l'Hyper-Threading).
 - LoadPercentage : Charge CPU instantanée (valeur entière de 0 à 100).
- 5. **Win32_Bios** :
 - SerialNumber : Numéro de série unique du matériel (*Service Tag*), clé primaire indispensable pour les outils de gestion de parc (CMDB).
 - SMBIOSBIOSVersion : Révision du micrologiciel UEFI/BIOS installée.

3. Optimisation Réseau : Le Filtrage à la Source via WQL

L'application tardive des filtres au sein du pipeline.

3.1 Le comparatif : Filtrage Client vs Filtrage Serveur

[APPROCHE CLIENT INEFFICACE : Where-Object]

Serveur Distant → [Instancie tous les disques] → [Sérialise en XML] → Réseau
(Transit massif)



Poste Client ← [Désérialise tout] ← [Where-Object { \$_.DriveType -eq 3 }] ←

[APPROCHE INDUSTRIELLE : Filtrage à la Source avec -Filter]

Serveur Distant → [Moteur WQL filtre dans le noyau] → Seul le disque C: transite → Réseau → Client

3.2 Syntaxe et règles du langage WQL

Le paramètre -Filter accepte une clause de restriction formulée en syntaxe **WQL** (*WMI Query Language*, sous-ensemble déclaratif du SQL-92).

Règles syntaxiques obligatoires en WQL :

- **Opérateurs de comparaison** : On utilise les opérateurs SQL classiques (=, !=, <, >, <=, >=) et **jamais** les opérateurs PowerShell (-eq, -ne, etc.).
- **Encadrement des chaînes de texte** : Les chaînes de caractères littérales doivent

obligatoirement être encapsulées entre **apostrophes simples** ('valeur').

- **Opérateurs logiques** : La combinaison de plusieurs conditions s'écrit avec les mots-clés en majuscules AND, OR, NOT.

Typologie des volumes (DriveType) dans Win32_LogicalDisk :

Valeur numérique	Signification système	Comportement en entreprise
2	Disque amovible (<i>Removable</i>)	Clés USB, cartes mémoires.
3	Disque fixe local (<i>Local Disk</i>)	SSD, HDD locaux, volumes virtuels de VM, LUNs SAN.
4	Lecteur réseau mappé (<i>Network Drive</i>)	Partages SMB connectés dans la session utilisateur.
5	Disque optique (<i>Compact Disc</i>)	Lecteurs CD/DVD physiques ou fichiers ISO montés.

Exemples de filtres à la source :

1. Isoler uniquement les disques fixes locaux (exclut clés USB et lecteurs CD)

```
Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType = 3"
```

2. Cibler chirurgicalement la partition système C:

```
Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DeviceID = 'C:'"
```

3. Combiner deux critères : disques fixes ayant moins de 10 Go d'espace (en octets)

```
$SeuilOctets = 10 * 1GB
```

```
Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType = 3 AND FreeSpace < $SeuilOctets"
```

4. Arithmétique Système et Manipulation des Dates

L'administrateur système ne doit jamais restituer des données d'instrumentation brutes sur un tableau de bord. Il doit maîtriser la conversion des grandeurs et le calcul différentiel de dates.

4.1 Calcul Dynamique de l'Uptime (Disponibilité Continue)

Sous l'ancien modèle WMI, LastBootUpTime renvoyait une chaîne de texte au format DMTF :

20260909061542.500000+120

Ce format textuel nécessitait des expressions régulières ou des méthodes de conversion complexes pour être comparé avec la date actuelle.

Sous CIM, LastBootUpTime est un véritable objet .NET natif de type **System.DateTime**.

Récupération de l'instance d'OS

\$OS = Get-CimInstance -ClassName Win32_OperatingSystem

La soustraction de deux objets DateTime génère un objet System.TimeSpan

\$TempsFonctionnement = (Get-Date) - \$OS.LastBootUpTime

Extraction des grandeurs

\$JoursEntiers = \$TempsFonctionnement.Days # Nombre entier de jours

\$TotalJours = [int]\$TempsFonctionnement.TotalDays # Durée totale exprimée en jours arrondis

Sécurité en production :

Un serveur Windows présentant un Uptime supérieur à 45 ou 60 jours indique généralement que les mises à jour cumulatives de sécurité mensuelles (*Patch Tuesday*) n'ont pas été finalisées par le redémarrage requis du noyau. L'Uptime est donc un indicateur direct de **dette de vulnérabilité**.

5. Sessions CIM Persistantes (New-CimSession)

Lorsque l'on doit auditer plusieurs composants sur un serveur distant (OS, disques, BIOS, CPU), exécuter des commandes Get-CimInstance -ComputerName \$Serveur successives force la machine cliente à ouvrir, authentifier et fermer une liaison réseau WinRM à chaque appel :

[SANS SESSION CIM PERSISTANTE : SURCÔÛT RÉSEAU ÉLEVÉ]

Get-CimInstance (OS) —► Négociation WinRM / Handshake —► Données —►
Fermeture Socket

Get-CimInstance (Disk) —► Négociation WinRM / Handshake —► Données —►
Fermeture Socket

Get-CimInstance (BIOS) —► Négociation WinRM / Handshake —► Données —►
Fermeture Socket

5.1 Mise en œuvre d'une session persistante

Une session CIM persistante maintient le canal WinRM ouvert, réduisant la latence et économisant les sockets réseau :

1. Ouverture d'un canal persistant vers le serveur cible

```
$Session = New-CimSession -ComputerName "SRV-FILE01" -ErrorAction Stop
```

```
try {
```

```
    # 2. Interrogation successive via le canal ouvert (extrêmement rapide)
```

```
    $OS = Get-CimInstance -CimSession $Session -ClassName Win32_OperatingSystem
```

```
    $Disk = Get-CimInstance -CimSession $Session -ClassName Win32_LogicalDisk -Filter  
"DeviceID='C:'"
```

```
    $Bios = Get-CimInstance -CimSession $Session -ClassName Win32_Bios
```

```
    $Sys = Get-CimInstance -CimSession $Session -ClassName Win32_ComputerSystem
```

```
}
```

```
finally {
```

```
    # 3. Libération impérative de la session réseau
```

```
    Remove-CimSession -CimSession $Session
```

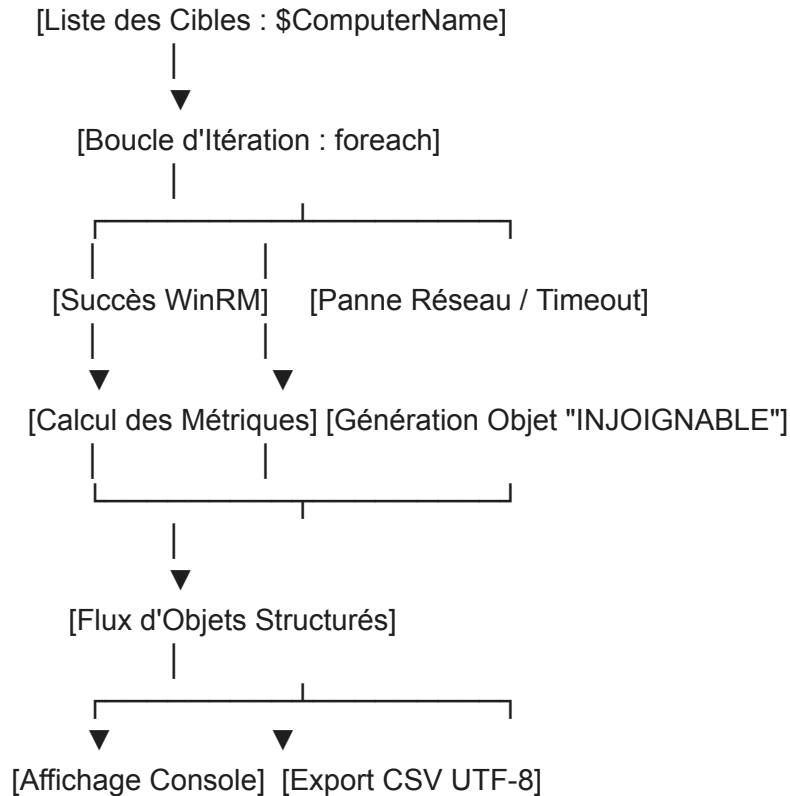
```
}
```

6. Patron de Conception : L'Inventaire Résilient de Production

En environnement réel, un script d'audit qui traite **50 serveurs** ne doit **jamais planter** si l'une des machines est éteinte, en cours de redémarrage ou isolée par un incident réseau.

6.1 Principes d'architecture logicielle

1. **Encapsulation par machine (try / catch)** : Chaque itération sur une machine cible doit être totalement étanche face aux exceptions levées par WinRM.
2. **Forçage d'interception (-ErrorAction Stop)** : Par défaut, un échec de connexion CIM produit une erreur non-terminante qui n'active pas le bloc catch. Le paramètre -ErrorAction Stop est obligatoire.
3. **Émission d'un objet normalisé de dégradation** : Si une cible ne répond pas, le script ne l'ignore pas : il émet un objet doté de la mention "INJOIGNABLE", permettant à l'équipe de supervision d'identifier précisément les hôtes en panne.



6.2 Modèle de consolidation en [PSCustomObject]

```

[CmdletBinding()]
param(
    [string[]]$ComputerName = @($env:COMPUTERNAME),
    [int]$SeuilDisqueLibrePct = 15
)

$RapportGlobal = foreach ($Machine in $ComputerName) {
    try {
        # Interrogation sécurisée
        $OS = Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $Machine -ErrorAction Stop
        $CS = Get-CimInstance -ClassName Win32_ComputerSystem -ComputerName $Machine -ErrorAction Stop
        $HD = Get-CimInstance -ClassName Win32_LogicalDisk -ComputerName $Machine -Filter "DeviceID='C:'" -ErrorAction Stop

        # Calculs arithmétiques
    }
}

```

```

$PctDisqueLibre = [math]::Round(($HD.FreeSpace / $HD.Size) * 100, 1)
$UptimeJours   = [int]((Get-Date) - $OS.LastBootUpTime).TotalDays

# Diagnostic logique
$Diagnostic = if ($PctDisqueLibre -lt $SeuilDisqueLibrePct) {
    "ALERTE_ESPACE_DISQUE"
} else {
    "CONFORME"
}

# Émission de l'objet complet
[PSCustomObject]@{
    Machine      = $Machine
    Fabricant     = $CS.Manufacturer
    Modele       = $CS.Model
    OS           = $OS.Caption
    Uptime_Jours  = $UptimeJours
    Disque_C_LibrePct = "$PctDisqueLibre %"
    Statut       = $Diagnostic
}
}
catch {
    # Gestion propre de la machine injoignable
    [PSCustomObject]@{
        Machine      = $Machine
        Fabricant     = "N/A"
        Modele       = "N/A"
        OS           = "N/A"
        Uptime_Jours  = "N/A"
        Disque_C_LibrePct = "N/A"
        Statut       = "INJOIGNABLE"
    }
}

# Restitution tabulaire
$RapportGlobal | Format-Table -AutoSize

```

7. Synthèse des Bonnes Pratiques de Métrologie

1. **Bannir définitivement Get-WmiObject** : Utiliser exclusivement les commandes *-CimInstance et *-CimSession.
2. **Filtrer systématiquement à la source** : Ne jamais rapatrier tous les disques pour appliquer un Where-Object DriveType -eq 3. Utiliser le paramètre -Filter "DriveType = 3" exécuté par le noyau du serveur.
3. **Se méfier des unités de mesure** : Vérifier systématiquement si la grandeur retournée par CIM est en octets (LogicalDisk, ComputerSystem) ou en kilo-octets (OperatingSystem).
4. **Préférer l'arithmétique native des dates** : Exploiter la nature System.DateTime de LastBootUpTime pour effectuer des soustractions directes avec Get-Date plutôt que de découper des chaînes textuelles.
5. **Garantir la persistance des exports** : Lors de l'exportation avec Export-Csv, déclarer impérativement -Delimiter ";", -NoTypeInfo et -Encoding UTF8 pour garantir l'interopérabilité avec les tableurs d'entreprise.